



A function-as-a-service middleware for decentralized collaborative edge computing

Catarina Gonçalves^{a,b}, José Simão^{a,c,*}, Luís Veiga^{a,b}

^a INESC-ID Lisboa, Lisboa, PT,

^b Instituto Superior Técnico, Universidade de Lisboa, Lisboa, PT,

^c Instituto Superior de Engenharia de Lisboa (ISEL / FIT), Lisboa, PT,

ARTICLE INFO

Keywords:

Function-as-a-service
Edge computing
Cloud computing
Volunteer computing
Peer-to-peer data networks

ABSTRACT

Function-as-a-Service (FaaS) emerges as a sophisticated cloud computing paradigm critically suited to processing the exponentially increasing data volumes generated by Internet of Things (IoT) infrastructures. Deploying computational models proximal to data generation sources addresses critical latency and bandwidth constraints inherent in edge-distributed applications. Edge computing environments present complex architectural challenges characterized by large-scale decentralized infrastructures and resource-constrained devices, which substantially impede contemporary Function-as-a-Service implementation strategies. This research introduces FaaS@Edge, a novel framework that leverages volunteered edge node resources discovered through the InterPlanetary File System (IPFS) network and deployed via Apache OpenWhisk. The proposed system addresses computational resource distribution challenges by enabling FaaS runtime deployments across heterogeneous edge infrastructure. Comprehensive experimental evaluation shows that FaaS@Edge introduces marginal latency during function submission while maintaining performance comparable to local OpenWhisk implementations. Empirical results demonstrate request success rates that approximate 98 % for function submission and invocation processes. These findings shows FaaS@Edge's potential as an efficient computational model for edge computing environments, characterized by low-latency performance and optimized resource allocation.

1. Introduction

Function-as-a-Service (FaaS) is an emerging paradigm to simplify native cloud systems and overcome its drawbacks by providing a simple interface to deploy event-driven applications that execute the function code, without the responsibility of provisioning, scaling, or managing the underlying infrastructure [1,3,4]. In the FaaS model, the management effort is detached from the responsibilities of the consumer, since the cloud provider transparently handles the lifecycle, execution, and scaling of the application. This model was originally proposed for the cloud mainly to address web request and event-oriented applications. However, it has since increased in popularity, being shown that apparently complex and traditionally batch-oriented applications, such as video transformation [2] and data analytics [5] can also be mapped to the FaaS paradigm. Further, it also has since been explored for deployments in geographically distributed systems [6].

With the expansion of the Internet of Things, the cloud has become an insufficient solution to respond to the growing amount of data transmitted and the variety of Internet of Things applications that require low latency and location-based deployments [7]. This led to

the introduction of the Edge Computing paradigm [8], with the aim of decentralized, energy-efficient, and resource-aware architectures to support the growing demand for real-time processing in IoT applications such as smart manufacturing and AI workloads [9,10]. The system presented in this paper, FaaS@Edge,¹ targets this vision by providing a decentralized Function-as-a-Service platform that operates efficiently on volunteer edge devices without relying on centralized cloud infrastructures. It has a peer-to-peer discovery mechanism, lightweight resource-aware scheduling, and local function execution.

In general, the Information and Communication Technologies (ICT) sector consumes 5–10 % of global energy production [11], with data centers and networks being significant contributors. Edge computing offers a promising alternative by localizing data processing, reducing unnecessary data transfers, and allowing more energy efficient computing models [8]. Importantly, edge computing isn't just a technological trend but a potential pathway to more sustainable digital infrastructure. By reducing data transfer, utilizing low-energy processors, and enabling

¹ <https://github.com/jsimaoisel/faas-edge>

* Corresponding author.

E-mail address: jose.simao@isel.pt (J. Simão).

<https://doi.org/10.1016/j.future.2025.108069>

Received 21 January 2025; Received in revised form 19 July 2025; Accepted 6 August 2025

Available online 8 August 2025

0167-739X/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

real-time energy management, it offers a promising approach to mitigating the environmental impacts of exponential digital growth. However, realizing this potential requires significant technological innovation. Researchers and engineers must develop new reference architectures, lightweight operating systems, secure middleware, and development tools that can operate efficiently at the edge of network infrastructures.

The intersection between Serverless FaaS and Edge Computing presents a captivating area of research and innovation, since the growing demand for low latency, real-time applications urges the need to explore the integration of FaaS in Edge Computing devices. At the same time, this integration also needs to address its inherent challenges, such as managing distributed architectures, optimizing resource allocation, and ensuring compatibility with the heterogeneous characteristics of edge devices.

Most of the current cloud service platforms still rely on centralized architectures and services that are not designed to operate on resource-constrained environments and the diverse variety of heterogeneous devices that characterize edge systems. In recent years, solutions have been explored to bring FaaS deployments to the edge of the network [6,12]. However, few have managed to achieve efficient resource provisioning and allocation [13], along with almost universal deployment, by using volunteered resources in a completely distributed and decentralized manner [14], in order to maximize resource utilization and meet the performance needs of edge applications.

The transition to distributed edge computing environments introduces significant challenges in resource scheduling and scalability, demanding algorithms that can dynamically allocate computational tasks across heterogeneous devices with varying capabilities and constraints. Effective resource optimization requires load-balancing mechanisms capable of efficiently distributing workloads while minimizing latency and energy consumption across diverse edge nodes. The main challenge lies in developing resilient scheduling frameworks that can efficiently manage task distribution, account for device heterogeneity, handle network instabilities, and maintain overall system performance in highly dynamic distributed computing landscapes.

This work makes the following key contributions:

1. It proposes a novel architecture for deploying Serverless and Functions-as-a-Service (FaaS) workloads at the network edge using volunteer computing resources.
2. It introduces *FaaS@Edge*, a new Function-as-a-Service framework that combines Apache OpenWhisk [15] and IPFS [16] (widely adopted technologies for their purposes of FaaS orchestration and decentralized content sharing) in order to support decentralized, resource-efficient execution of serverless functions across peer-to-peer edge nodes.
3. It demonstrates, through implementation and evaluation, the feasibility and performance characteristics of volunteer-based edge computing for FaaS, providing a decentralized alternative to cloud-based models.

This work builds on and extends our previous research [17] by enhancing the scalability of the system, demonstrating its applicability to scenarios with variable network conditions, and assessing application to devices with limited resources, such as the Raspberry Pi 3 model B.

The rest of the paper is organized as follows: Section 2 introduces the background and context on relevant domains of the work. Section 3 details the design and implementation of the FaaS@Edge runtime, including its integration of Apache OpenWhisk and IPFS, and the mechanisms for function discovery and coordination across volunteer edge nodes. Section 4 presents a detailed evaluation of the system's performance, scalability, and resource efficiency in volunteer-based edge environments. Section 5 reviews specific related work across Function-as-a-Service (FaaS), edge computing, and peer-to-peer systems, highlighting the research gaps that FaaS@Edge addresses. Finally, Section 6 summarizes our contributions and discusses potential avenues for future work.

2. Background and context

In this section, we introduce the background of the two key domains our framework addresses: the Serverless model and Edge Computing. For each we present a taxonomy that includes the main analysis dimensions in such settings.

2.1. Serverless model

Although the FaaS model, introduced by Amazon as Lambda functions, is a relatively recent Cloud Computing paradigm, it has been extensively studied, enabling automatic execution of function code at a fine-grained level in response to events without requiring virtual machine provisioning or system maintenance. Fig. 1 presents a taxonomy for classifying FaaS models. Jonas et al. [3] provide a comprehensive analysis of *Serverless Computing* in comparison to Virtual Machine-based solutions, highlighting associated challenges and potential future directions. Mohanty et al. [18] evaluate and compare the features of existing open-source FaaS frameworks, while Wen et al. [19] analyze and compare the characteristics of commercial FaaS platforms. The taxonomy defines the key attributes that distinguish FaaS offerings, focusing on four primary dimensions: Computing Environment, Development, Deployment, and Runtime. Computing Environment – FaaS platforms are categorized into:

- i) Commercial Platforms: Proprietary services offered by providers such as AWS Lambda, Google Cloud Functions, Microsoft Azure Functions, and IBM Cloud Functions. These typically operate on a pay-as-you-go model but may lead to vendor lock-in;
- ii) Open Source Frameworks: Freely available and modifiable solutions such as OpenWhisk, Kubeless, Fission, and OpenFaaS. These offer greater customization and avoid vendor constraints.

Development – Involves several important attributes:

- i) Programming Languages: Commonly supported languages include Python and Java.
- ii) Function Triggers: Events that invoke functions, such as HTTP requests, storage events, and timers.
- iii) Package Size Limits: Restrictions on deployment packet size (e.g., 50 MB for AWS Lambda), which can affect cold start latency.

Deployment – This phase includes:

- i) Deployment Methods: Approaches like Docker containers or integration with Kubernetes;

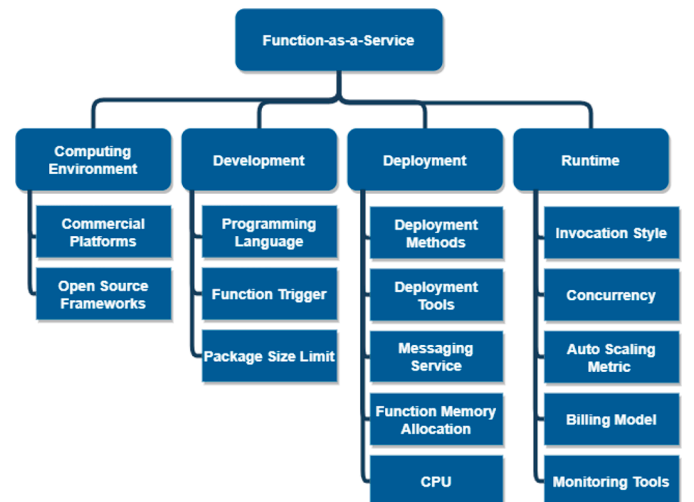


Fig. 1. Function-as-a-service taxonomy.

- ii) **Deployment Tools:** Interfaces such as command-line interfaces (CLI), application programming interfaces (API), and software development kits (SDK);
- iii) **Messaging Services Integration:** Examples include services like Amazon SQS;
- iv) **Function Configuration:** Customization of memory and CPU allocation, typically with CPU scaling proportionally to memory.

Runtime – Characteristics of the runtime include:

- i) **Invocation Styles:** Either *Synchronous* or *Asynchronous* execution modes.
- ii) **Concurrency:** The maximum number of simultaneous function executions;
- iii) **Auto Scaling Metrics:** Triggered by factors such as Queries Per Second (QPS) or Requests Per Second (RPS);
- iv) **Billing Models:** Usually based on per-request or duration-based billing;
- v) **Monitoring Tools:** Performance and usage tracking through tools like Prometheus.

2.2. Edge computing

This section presents a taxonomy for classifying Edge Computing models (Fig. 2). Existing literature offers various classifications, including layered architectures [20], orchestration frameworks [21], and resource management systems [22]. We highlight key design principles, architectural features, and characteristics that distinguish these models.

Architecture – Edge Computing architectures can be centralized, relying on a controller for resource management, or decentralized, employing hierarchical tiers or peer-to-peer coordination to distribute responsibilities among nodes.

Computing Environment – Edge Computing environments vary across Virtual Machines, Containers, Processes, and Browsers, each offering distinct isolation, scalability, and performance characteristics based on the underlying hardware and software.

Resource Ownership – Defines who owns the physical devices powering the Edge Computing system. Ownership models include:

- i) **Volunteer Devices:** Personally owned devices contributed by the public, incentivized through mechanisms like reputation or virtual currency (e.g., Filecoin);
- ii) **Infrastructure Owner:** Entities like ISPs, mobile providers, or on-premises organizations that control infrastructure resources;
- iii) **Combination of Ownership:** Hybrid models where higher tiers are owned by infrastructure providers and lower tiers involve volunteered devices.

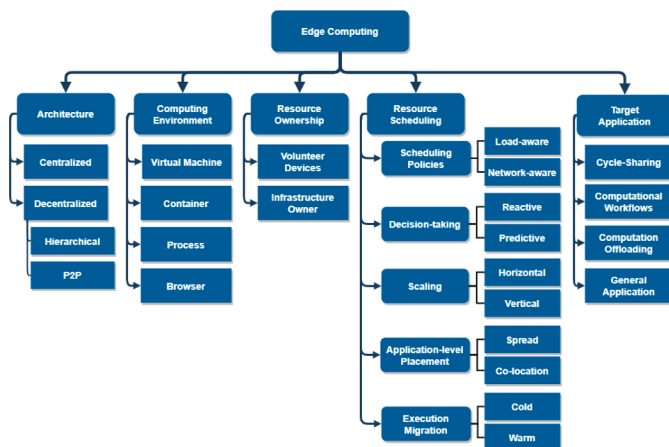


Fig. 2. Edge computing taxonomy.

Resource Scheduling – The process of provisioning and allocating resources, accounting for latency, computational overhead, and coordination delays.

Scheduling Policies – Execution placement strategies, typically:

- i) **Load-aware Policies:** Focus on balancing computational loads;
- ii) **Network-aware Policies:** Aim to minimize latency and maximize bandwidth efficiency.

Application-level Placement – Determines where to place application components within the network, which includes

- i) **Spread Placement:** Minimizes bottlenecks by distributing components across nodes;
- ii) **Co-location:** Reduces latency by placing related components on the same or nearby nodes.

Execution Migration – Involves relocating an active service from one node to another:

- i) **Cold Migration:** Stops the service and restarts it elsewhere;
- ii) **Warm Migration:** Transfers the service state while running, reducing downtime.

2.3. Applications, economic viability and sustainability

The deployment of functions at the edge, in a decentralized and collaborative environment, introduces the possibility of developing applications that are lightweight, event-driven, and sensitive to latency or bandwidth constraints. Unlike traditional centralized FaaS systems, our goal was to design a system to support the sharing of prosumer-based resources between heterogeneous and potentially ephemeral devices. This context is especially meaningful for scenarios such as the following [12,23–25]:

- a) **IoT Data Preprocessing:** In scenarios such as smart homes, traffic monitoring, or environmental sensing, edge devices can execute lightweight preprocessing tasks (e.g., filtering, aggregation, anomaly detection) close to data sources, reducing bandwidth usage and response times.
- b) **Local media processing:** In scenarios where raw data, such as sensor outputs or images, is transformed or analysed at the edge before being stored or transmitted. This reduces payload sizes and improves responsiveness, especially in low-bandwidth or high-latency environments.
- c) **Decentralized querying of structured data:** Data is intermittently updated and accessed based on localized criteria. In such settings, deploying search logic close to the data source reduces the load on central infrastructures and improves privacy.
- d) **Apps for Web3:** Distributed Applications for what is known as the Web 3.0, dApps, could be deployed as functions in a truly decentralized form, while relying on some trusted blockchain infrastructure for other services like identity and billing management.

Regarding economic viability, compared to traditional cloud-based approaches, our system aims to utilize existing resources, relying on underused computing capacity from volunteers or local organizations. Also doing so, community clouds [26] can avoid vendor lock-in using open-source technologies, which improves flexibility and long-term cost control. Incentive mechanisms (e.g., credits, or reciprocal use) can be layered atop the framework to foster continued participation without requiring direct monetary exchanges.

When concerning environmental sustainability, our work can contribute to improvements in eco-related metrics in line with recent work, such as Computational Carbon Intensity [27] (CCI) and Software Carbon Intensity [28,29] (SCI). These aim at assessing the complete life-time carbon impact of systems (both resulting from operation and also

embodied emissions in the hardware). Such works provide us with arguments to promote the reuse of existing devices already deployed (even when more limited in resources, but suitable for FaaS execution) in voluntary computing at the edge, amortizing the embodied emissions of such hardware, when performance is not critical.

3. Architecture and implementation details

FaaS@Edge is a decentralized middleware framework designed to support Function-as-a-Service (FaaS) execution across volunteer edge devices. It aims to reduce execution latency, improve utilization of distributed resources, and enhance content availability in heterogeneous and dynamic edge environments. The system architecture requires each participating node to run a minimal set of components that enable discovery, coordination, and execution of serverless functions, as illustrated in Fig. 3. At a high level, each node includes three main components:

1. The middleware daemon of FaaS@Edge, responsible for orchestrating function submission, discovery, and scheduling decisions;
2. A peer-to-peer storage and content distribution layer to enable decentralized publishing and retrieval of function code and resource offers;
3. In each node that offers resources, a serverless function execution environment capable of running incoming function requests.

In this paper, the FaaS@Edge assumes that components for items 2 and 3 are the Intra-Planetary File System and the standalone version of OpenWhisk, respectively. OpenWhisk [15] is an open source widely used example of a serverless framework that provides function execution capabilities without having to manage the servers and underlying infrastructure. In OpenWhisk, functions that execute code are called Actions and can be written in multiple programming languages. Their execution can be driven by events, called triggers, coming from a variety of sources, or manually, using the designated CLI or REST API. However, the architecture and algorithms described in the following sections remain extensible to alternative components that fulfil the same functional roles.

3.1. Distributed architecture

The FaaS@Edge architecture is designed using the IPFS peer-to-peer framework [30]. The InterPlanetary File System is a peer-to-peer distributed file system. At its core, IPFS operates by content addressing rather than location-addressing: each file and all blocks within it are given a unique cryptographic hash, ensuring immutability and integrity. These content identifiers (CIDs) enable distributed data retrieval. It

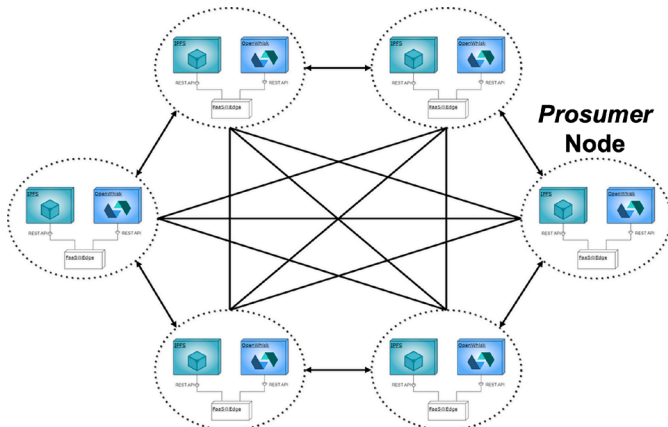


Fig. 3. FaaS@Edge participant node's overview architecture.

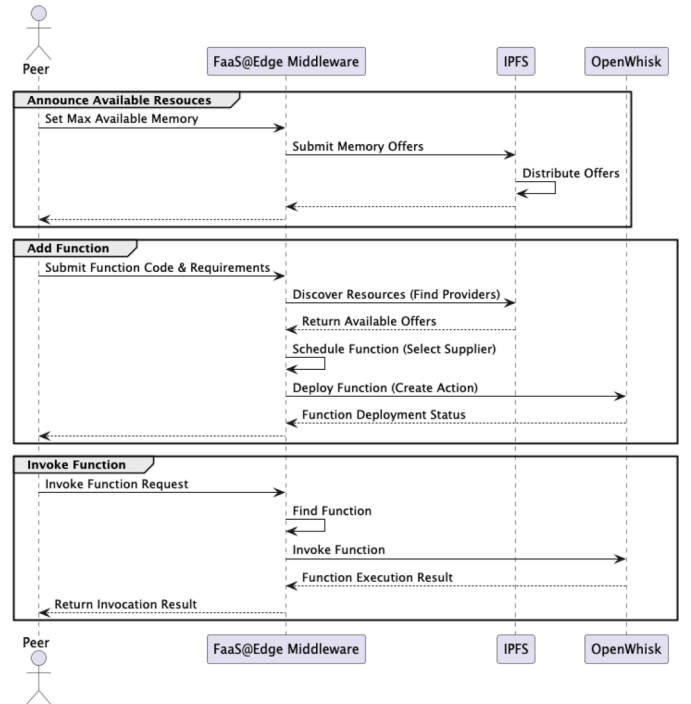


Fig. 4. Sequence diagram of the three major operation: offer resources; submit function and invoke function.

leverages Kademlia-based Distributed Hash Table (DHT) for content discovery and its integration with protocols like *libp2p* for secure and modular networking make it a robust foundation for decentralized applications and resilient web architectures.

This setup enables efficient lookup, routing, and retrieval mechanisms critical for large-scale content distribution, with built-in caching enhancing performance. Each FaaS@Edge node is uniquely identified by a PeerID, a SHA-256 multihash of the node's public key, and operates in a distributed and decentralized manner to discover resources. Nodes broadcast resource availability, such as memory, by publishing files tagged with specific strings and Content Identifiers (CIDs) on IPFS, allowing other nodes to locate and deploy functions efficiently.

FaaS@Edge introduces over IPFS the concept of prosumers, that is, nodes that can act as both resource suppliers and consumers. Suppliers volunteer their resources, such as memory, by running the OpenWhisk component and specifying the maximum memory they are willing to offer during initialization. These nodes advertise their resources on the network, enabling decentralized resource sharing. Meanwhile, all nodes, regardless of their role as suppliers, can also send function deployment requests, acting as clients seeking resources. During initialization, each node precomputes the CIDs of possible resource offers (ranging from 128MB to 512MB in powers of 2) using IPFS' *only-hash* option with the *add* command. These CID are stored for use in supply and discovery algorithms, ensuring seamless operation in both supplier and consumer roles.

Fig. 4 illustrates a sequence of interactions in a Peer node of the FaaS@Edge Middleware, IPFS and OpenWhisk. It is divided into three main processes: Announce Available Resources, Add Function, and Invoke Function. In the first process, a peer sets the maximum available memory, which the middleware uses to create and submit memory offers. These offers are then distributed via IPFS to make them discoverable by the system.

In the Add Function phase, a peer submits to the middleware the function code along with resource requirements. The middleware searches for a compatible offer through IPFS, receives available offers, and selects a suitable supplier. The function is then deployed on Open-

Whisk. In the final phase, Invoke Function, a peer can request the function execution. The middleware locates the function and forwards the invocation to OpenWhisk. After execution, the result is returned to the peer, completing the function-as-a-service workflow at the edge. With the overall system workflow and interactions defined in the sequence diagram, we now describe to the core logic of the system in the following section.

3.2. Resource scheduling algorithms

The following data structures are used for the resource scheduling algorithms:

- **Supplier's Active Offers Map:** Maintained by each node offering resources, this map tracks the number of offers for each resource value.
- **Available Offer:** Stores the resources and supplier IP address associated with an offer discovered in IPFS.
- **Function's Configuration:** Contains the necessary details for submitting a function, including the source code's content identifier (CID), function name, runtime type, and required resources.
- **Function's Status:** Includes the function's configuration, the supplier node's IP address where the function is deployed, and the submission status.
- **Invocation's Arguments:** Holds the parameters required to invoke a function, such as the function name, its parameters, and the invocation result indicating if a response is expected.
- **Invocation's Result:** Contains the invocation arguments, response, and status.

3.2.1. Supplying

Algorithm 1 is executed by a supplier node upon its initial integration into the system or whenever its resource availability fluctuates, either due to the allocation of a function or by freeing resources following a deployment failure. The node starts by calculating the amount of resources currently in use, given the maximum value of resources it is willing to provide and the current value of free resources it has. Then, all active offers are removed in order to calculate the offers that match the current resource availability. This is achieved by making the IPFS client to remove the offer file's *pin* per each size of active offer, the remaining offers are simply decremented in the supplier's active offers map. Given the distributed nature of IPFS and its caching capabilities, there is no direct way to delete a file, only to *unpin* it from storage and let the garbage collector reclaim it.

Algorithm 1: Supplier's create offer algorithm used to create an offer in the system.

```

Data: supplierActiveOffersMap, IPFSCClient
Result: NewOffer
Function CreateOffer(offerResources):
  if supplierActiveOffersMap[offerResources.Value] < 1
  then
    offerString ← GetResourcesString(offerResources)
    ok ← IPFSCClient.Add(offerString)
    if ok = false then
      return Error("Unable to create offer")
  /* Only needs to add to IPFS if there are no
  active offers of that memory value, otherwise,
  just creates the new offer to add to the map.
  */
  newOffer ← NewOffer(offerResources)
  return newOffer

```

The described algorithm is used when a supplier decides to make offers in the system and is part of the complete supply protocol detailed in **Algorithm 2**. This protocol is run by a supplier node when:

- The node first joins FaaS@Edge and wants to start providing its resources;
- The node is submitting a user function in OpenWhisk upon request, thus consuming its resources and causing the need to update its offers according to its new resource availability values;
- The node failed to submit a user function in OpenWhisk upon request, thus needing to release the resources selected to be consumed and causing the need to update its offers in order to match its new resource availability.

Algorithm 2: Full supply algorithm.

```

Data: supplierActiveOffersMap, supplierOfferPlan
Function SupplyResources(freeResources, maxResources):
  usedResources ←
    ResourcesInUse(freeResources, maxResources)
  removeSupplierOffers()
  offerCount, offerSize ←
    supplierOfferPlan.CalculateOffers()
  foreach offerCount, offerSize do
    newOffer ← CreateOffer(offerSize)
    supplierActiveOffersMap.Add(newOffer)

```

3.2.2. Discovering

The discovery process begins after supplier nodes publish their available resources as offers in the system. This phase involves locating a suitable supplier node to submit a user's function request. **Algorithm 3** outlines the resource discovery process. Upon receiving a function submission request with specified resource requirements, the node determines the smallest predefined memory size that satisfies the request. This step identifies the appropriate memory tier and its corresponding CID. The node then uses the IPFS client's *Find Providers* operation to search the network for peers offering resources associated with that CID. This involves querying the distributed hash table (DHT) maintained by IPFS, which returns peer records containing the IPFS addresses of the providers. For each provider found (up to 20 by default), the system creates a new *Available Offer* object. This object includes details about the offered resources and the supplier node's IP address, extracted from the provider's IPFS address.

Algorithm 3: Resource's discovery algorithm.

```

Data: IPFSCClient
Result: availableOffers
Function DiscoverResources(neededResources):
  availableOffers ← ∅
  fittedResources ← FitResources(neededResources)
  resourcesCID ← GetResourcesCID(fittedResources)
  providers ← IPFSCClient.FindProviders(resourcesCID)
  foreach p ∈ providers do
    newAvailableOffer ←
      NewAvailableOffer(fittedResources, p.IPAddress)
    availableOffers.Add(newAvailableOffer)
  return availableOffers

```

3.2.3. Scheduling

This section explains the scheduling phase, which follows the resource discovery process. During scheduling, users submit function requests via the CLI, which interacts with FaaS@Edge's daemon. Upon receiving a request with the function's configuration, the system executes the *Schedule* function (**Algorithm 4**) to allocate the function to a suitable supplier node.

The *Schedule* function first determines the required resources from the function's configuration. It then calls the *DiscoverResources* function to retrieve available offers. These offers are randomly sorted to distribute the load evenly across supplier nodes. The system iterates over the sorted offers, sending a *SubmitFunction* message to the supplier node, which includes the function's configuration, the selected offer, and the client's IP address.

If no supplier can host the function, the system returns an error to the user, indicating a failure of deployment. Failures may occur due to concurrent attempts to use the same supplier's resources or outdated information in the IPFS network about a supplier's availability. Users can retry the request manually in such cases, or with scripting this can be done automatically, as it could also happen with a cloud provider API. If successful, the node stores the deployed function's configuration and the supplier's IP address in a map, indexed by the function name, for future invocation requests.

Algorithm 4: Algorithm to schedule function in supplier node.

```

Function Schedule(fnConfig):
    resNeeded ← fnConfig.Resources
    availOffers ← DiscoverResources(resNeeded)
    availOffers ← RandomOrder(availOffers)
    foreach offer in availOffers do
        fnStatus ← SubmitFunction(fnConfig, offer, self.IP)
        if fnStatus = ok then
            deployedFn ←
                DeployedFn(fnConfig, offer.SuppIP)
            functionsMap.Add(deployedFn)
            return fnStatus
    return Error("Unable to schedule function")

```

The scheduling process on the supplier node is outlined in Algorithm 5. When the supplier node receives a function submission request, it begins by reserving the resources specified in the offer and updating the available resources. Next, the supplier invokes the OpenWhisk component, passing the function's configuration to retrieve the function's source code from IPFS using its CID. The function (referred to as an "action" in OpenWhisk) is then created based on the specified configurations.

Algorithm 5: Supplier node's algorithm to deploy a function in OpenWhisk.

```

Data: supplier, OpenWhiskClient
Function
    DeployFunction(functionConfig, offer, clientNodeIP):
        if supplier.UseResources(offer.Resources) != ok then
            return Error("Resources not valid")
        functionStatus ←
            OpenWhiskClient.InsertFunction(functionConfig)
        if functionStatus = ok then
            localFunction ←
                NewLocalFunction(functionConfig, clientNodeIP)
            localFunctionsMap.Add(localFunction)
            return functionStatus
        else
            supplier.ReleaseResources(offer.Resources)
            return Error("Unable to submit function")

```

If the function is successfully created, the supplier node stores the function details in a local map, organized by client node, to facilitate future invocations. The supplier then notifies the client node of the successful deployment. In case of failure during function creation, the supplier releases the reserved resources and sends an error message to the

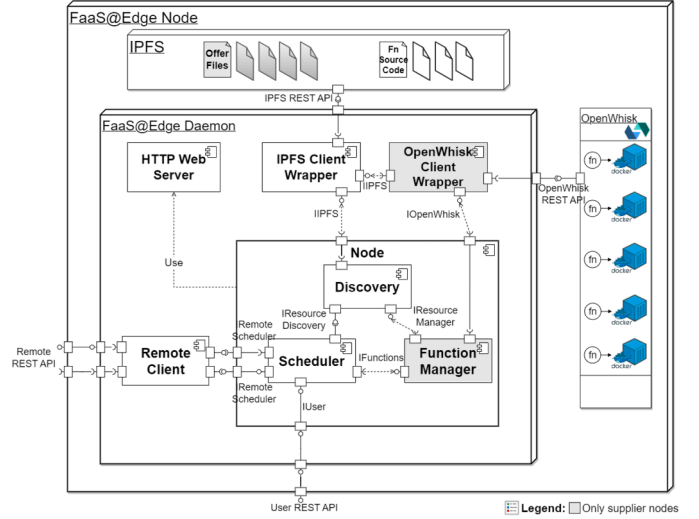


Fig. 5. FaaS@Edge's components and interfaces.

client node, indicating the deployment issue. Once again, this can be subject to retry, either manual or automated.

3.3. Software components

Fig. 5 provides an overview of FaaS@Edge's node software components, interfaces, and their relationships. Note that only the supplier nodes need to include the OpenWhisk Client Wrapper and Function Manager components. The main components are:

Node: A central component that initializes all other components using user-provided configuration through the CLI tool. It exposes scheduling services to users and other nodes via a REST API.

Scheduler: Handles function deployments and invocations. It provides interfaces for user requests and remote node communication, enabling function deployment and invocation. It also works with the Discovery component to locate resources for deployments.

Discovery: Executes resource discovery algorithms to identify resources from other nodes and manage the local node's resource offers based on the offering plan. It interacts with the IPFS Client Wrapper to add offers to IPFS, query the DHT, and retrieve CIDs for resource values.

Function Manager: Oversees function deployments on the local OpenWhisk platform via the OpenWhisk Client Wrapper. It interfaces with the Scheduler to handle function submissions and invocations while validating and releasing resources through the Discovery component in case of errors.

IPFS Client Wrapper: Provides a simplified interface for interacting with IPFS by wrapping its HTTP RPC API. This abstraction ensures middleware-level integration without direct reliance on IPFS' core API commands.

OpenWhisk Client Wrapper: Offers an abstraction over the OpenWhisk API using the Go client library. It enables the Function Manager to insert, invoke, and delete functions while enforcing memory limits for functions, as defined during their insertion.

HTTP Web Server: Hosts REST API endpoints and routes requests to the appropriate FaaS@Edge components. It is initialized by the Node component when the user issues a start command.

Although we selected OpenWhisk and IPFS due to their widespread usage, the architecture could be adapted to other open-source FaaS frameworks (e.g., OpenFaaS, Knative); P2P structured overlays, e.g., Chord, CAN; P2P storage, e.g., Freenet. Currently, at each the FaaS@Edge node, the Kubo version is installed, a Go-based implementation of the IPFS protocol. FaaS@Edge implementation was also written in the Go language. Regarding Openwhisk, the system is using at each node the standalone version, a deployment mode of Apache OpenWhisk which

runs all core components within a single host, typically using Docker containers. This is much less resource-intensive than the full release, simplifies setup greatly and reduces communication overhead, making it suitable for edge devices like Raspberry Pi or small VMs. While not optimized for large-scale production use, it preserves the full programming model and API of OpenWhisk, enabling deployment in decentralized edge networks.

3.4. Command line interface

FaaS@Edge provides a CLI tool to consume the REST API, similar to OpenWhisk's CLI tool, that allows users to perform the following operations: *Start* - Start running a new FaaS@Edge node; *Exit* - Shut down the instance node; *Submit* - Submit a user function in FaaS@Edge.; *Invoke* - Invoke a function previously submitted in FaaS@Edge.

FaaS@Edge provides a CLI tool to consume the REST API, similar to OpenWhisk's CLI tool, that allows users to perform the following operations:

- *Start* - Start running a new FaaS@Edge node. This command should be issued to run as a daemon since it is in charge of all FaaS@Edge's operations. The command can specify two flags: an integer flag detailing the maximum amount of memory (in MB) that the node is willing to offer, and a boolean flag specifying that the node is running the OpenWhisk application.
- *Exit* - Shut down the instance node.
- *Submit* - Submit a user function in FaaS@Edge. The command should specify the CID in IPFS of the function's source code, the function's name, the type of programming language, and the memory limit that it can allocate.
- *Invoke* - Invoke a function previously submitted in FaaS@Edge. The command should specify the function's name, the function's parameters in JSON format, and whether the invocation should return a result or not.

4. Evaluation

To evaluate the FaaS@Edge system, we ran experiments on different hardware tiers:

- a cluster of 15 cloud VMs (each with 2 vCPUs and 2 GB RAM), together with a remote client node (2 vCPUs, 4 GB RAM) located in a different region (used in [Sections 4.1–4.5](#)).
- two other similarly configured clusters, of 20 and 40 cloud VMs, where used to address geo-distribution, where we used Linux traffic-shaping (tc) to emulate realistic edge networks by varying round-trip latency, jitter and packet loss across runs (used in [Section 4.6](#)).
- an actual Raspberry Pi 3 Model B node (quad-core ARM Cortex-A53 @ 1.2 GHz, 1 GB RAM), used in [Section 4.7](#).

During testing, a private IPFS network was configured for FaaS@Edge instances instead of using the default global IPFS network. This approach ensures privacy and restricts data access to known peers. The setup involved modifying each IPFS node's bootstrap list by replacing the default entries with the IP address and peer ID of the private network's bootstrap nodes, alongside referencing the network's swarm key. In practical scenarios, these values can be provided by a network administrator when a node requests to join, similar to how IPFS maintains a list of popular content providers.

To evaluate our system, we developed FaaS workload functions in the Go language, adapted to our prototype and designed to simulate some common FaaS scenarios [4,31]:

- a) *Content Hashing* - Accepts data content as a parameter and computes its SHA256 hash. If requested, the resulting hash is returned to the user;

- b) *Database Query* - Allows the initialization of an in-memory database containing a library's book information stored in JSON format. Users can query the database for a specific book by providing its International Standard Book Number (ISBN) as input;
- c) *Image Transformation* - Takes a public image URL as input, retrieves the image data via HTTP, applies a vertical flip transformation, and returns the modified image data in base64 format.

Our evaluation examines the overhead introduced by the distributed architecture and algorithms of the FaaS@Edge middleware compared to a local OpenWhisk deployment, focusing on its feasibility and performance on edge devices. The dataset includes submission and invocation requests for three function types, with memory allocation sizes of 128MB, 256MB, and 512MB, adhering to OpenWhisk's allowed limits. During testing, all client nodes issue an equal number of parallel requests, and the supplier nodes collectively provide sufficient resources to handle all requests.

The evaluation aimed to measure the overhead introduced by FaaS@Edge's distributed architecture and algorithms compared to a local OpenWhisk deployment and assess its feasibility and performance on edge devices. The following metrics were used:

- i) *Function latency*: Measures request execution time, discriminating between function submission and function invocation, isolating OpenWhisk execution to capture resource discovery delays;
- ii) *Bandwidth consumption per node*: Evaluates the network usage to ensure it remains feasible for edge nodes;
- iii) *CPU usage per node*: Assesses whether edge nodes' processors can handle the load;
- iv) *Memory usage per node*: Examines the RAM requirements for running the FaaS@Edge middleware;
- v) *Request success rate*: Evaluates the effectiveness of resource discovery and scheduling in finding suitable supplier nodes;
- vi) *The scalability bottlenecks of concurrent function submissions on a local deployment of the OpenWhisk*;
- vii) *The impact of changes on networking conditions, such as communications between distributed geo-location where latency, jitter and packet loss can increase*;
- viii) *The feasibility of running the middleware in a more resource-restrained device, such as an hardware like Raspberry Pi*.

4.1. Function latency

One of the scheduling options in FaaS@Edge is a scenario where the supplier node provides the same number of offers for each size, without exceeding the node's maximum resource capacity. We used this approach to measure how the metrics identified performed in six deployment setups:

- i) One client node and one supplier node on the same machine;
- ii) One client node and one supplier node on remotely distant machines;
- iii) Five nodes with two supplier nodes and three client nodes on the same machine;
- iv) Ten nodes with five supplier nodes and four client nodes on the same machine, and a client node on a remote machine;
- v) Fifteen nodes with eight supplier nodes and six client nodes on the same machine, and a client node on a remote machine.
- vi) Local deployment of OpenWhisk on a single node instance;

The remainder of this section presents the results of our evaluation.

This section evaluates two distinct sources of latency that influence the performance of our system: submission latency and invocation latency. Submission latency refers to the time required by the platform to deploy a function upon request, which involves selecting a suitable execution node from a set of available offers. This process involves coordination between resource management and scheduling components and reflects the overhead associated with dynamic placement decisions. In

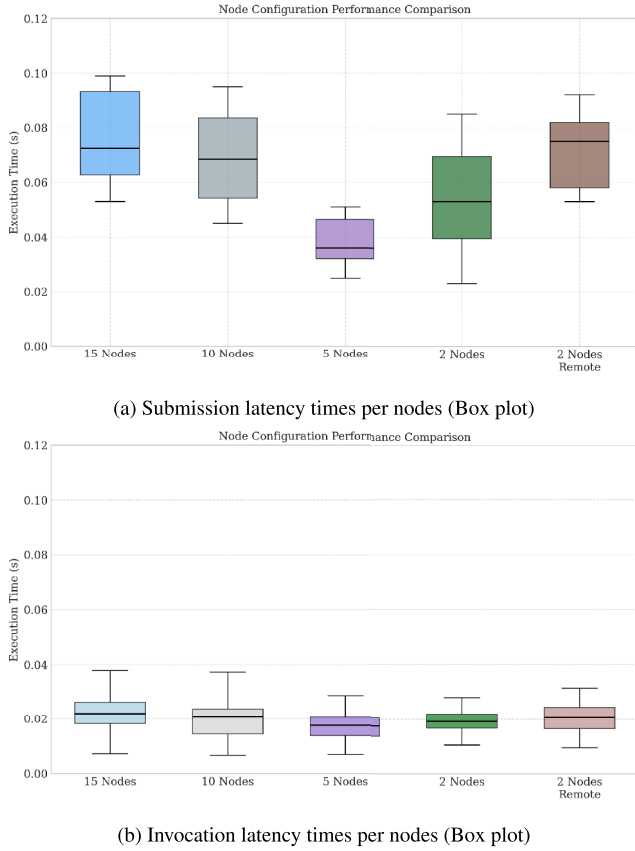


Fig. 6. Submission and invocation latency for different nodes configurations.

contrast, invocation latency denotes the time taken to execute a function that has already been deployed and registered within the OpenWhisk instance assigned to the selected node. It captures the responsiveness of the execution under the assumption that no additional deployment steps are required. Note that before the first invocation the function needs to be created as an action in the local OpenWhisk instance. Because this happens only once, despite the number of invocations, this time is not considered here.

Fig. 6(a) shows the distribution of submission latency times for the deployments described earlier. The latency was measured from the moment the client nodes sent the submission requests to when a response was received, excluding the time taken by the supplier node to create the function in OpenWhisk. The observed latency values ranged between 0.025 and 0.1 s. Lower latency values were associated with the 2-node and 5-node deployments, while higher latencies were observed in the 15-node deployment and when the client was located on a remote machine.

Fig. 6(b) presents the request latency times in deployments with varying numbers of nodes, excluding the execution time of the OpenWhisk function. The results showed a highly consistent latency, with all times falling within a narrow 0.04s interval. No significant correlation was found between number of nodes and latency. This was expected, as invocation requests do not incur the overhead of resource discovery and scheduling algorithms, since required information is stored on the supplier node during initial function submission.

Fig. 7(a) summarizes the average total submission times and submission latency times for each type of FaaS function during execution requests. The total submission times, measured from request initiation to obtaining the response, were approximately 0.11 s, while latency times ranged between 0.058 and 0.065 s. These results were consistent across function types since the function's source code does not influence resource discovery, supplier availability, or submission time of the actual

function in OpenWhisk. The memory values specified in the submission requests (128MB, 256MB and 512MB) played an important role in selecting an appropriate provider. However, as shown in Fig. 7b–d, all memory configurations resulted in similar overhead times across function types. This indicates that the offering strategy effectively distributed resources of varying sizes.

Fig. 8(a) presents the average total invocation and latency times across all memory sizes, while Fig. 8b–d show the comparison for each individual FaaS workload function. The results exclude the data for the first and, in some cases, the second invocations, as these are influenced by cold start effects, which occur when there is no pre-existing container at the time of the invocation request. That would also happen in a local deployment and container set-up could take longer than the actual invocation. This could dwarf any relative comparison between local executions and executions through FaaS@Edge, and thus hide any potential FaaS@Edge induced overhead. Therefore, the times presented are based on *warm start* invocations to normalize the averages.

The data reveal a significantly longer total invocation time for the image transformation function, which has higher CPU demands, compared to the content hashing and data base query functions. This increased time is spent during execution in OpenWhisk, as all functions show similar invocation latency times. The memory allocation limits defined by different memory values in the requests do not significantly impact the total or latency invocation times, as shown across Fig. 8. All functions are able to execute within the memory limits, and the invocation times does not exhibit a clear trend of improvement or degradation relative to the memory settings.

Unlike submission times, the remote client experienced only a 2.9% increase in total invocation time, suggesting that while physical distance can affect IPFS' lookup protocol during resource discovery, it has minimal impact on invocation execution times, maintaining acceptable network throughput. The results show that FaaS@Edge is slower than local deployment, with submissions taking 90.9% longer and invocations 25.5% slower. However, this relative delay is acceptable, as FaaS@Edge allows less powerful edge nodes to operate independently of cloud providers. The minimal performance loss is particularly beneficial given that invocation requests are likely more frequent than submission requests in FaaS@Edge.

4.2. Bandwidth consumed per node

Fig. 9 illustrates the bandwidth consumption patterns in different node deployments. The results show that bandwidth variation decreases as the number of nodes increases, with median values ranging from 8659 to 9604 bytes, indicating an improved load distribution at scale. Periodic bandwidth spikes of approximately 3000 bytes were observed, with the image transformation function exhibiting additional incoming traffic due to HTTP image retrieval operations. In particular, bandwidth consumption per node did not show a significant correlation with request volume. These findings suggest that the bandwidth requirements for edge device operations remain within acceptable parameters and do not adversely affect node performance. Fig. 10 illustrates the bandwidth consumption as a function of memory allocation for the three types of workload. Results indicate uniform bandwidth usage across all function types at 256MB, as well as for database query functions at 128MB and content hashing functions at 512MB. As anticipated, the data confirms the absence of a direct correlation between allocated memory size and bandwidth consumption at the node level.

4.3. CPU and memory usage per node

The CPU and Memory used per node metrics were retrieved periodically over time on both supplier and client nodes, during each test execution. The average CPU usage observed in supplier nodes and client nodes for each deployment gradually decreased from 5.61% to 2.31%

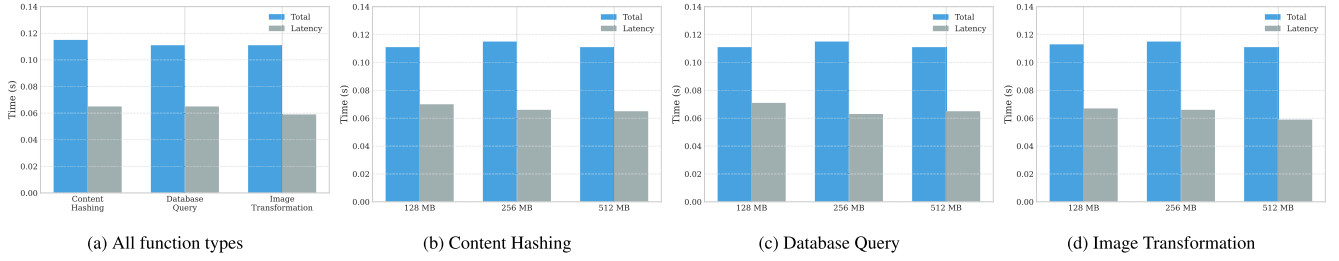


Fig. 7. Submission times per function memory value: (a) overall (b) Content hashing, (c) Database query, (d) Image transformation.

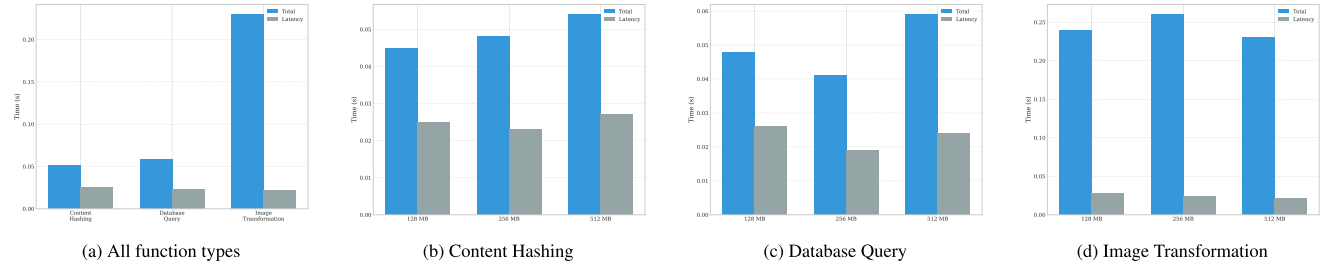


Fig. 8. Invocation times per function memory value.

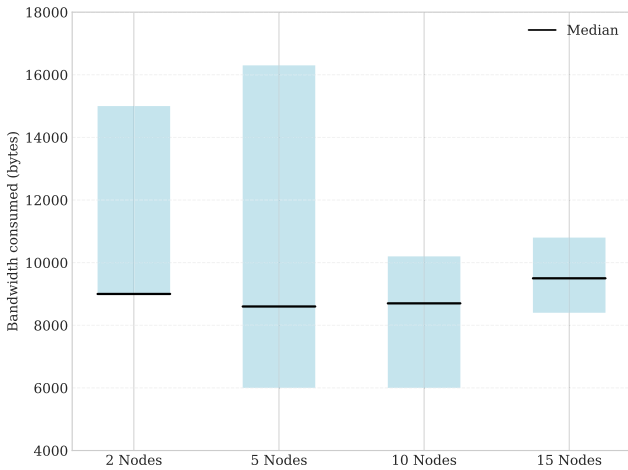


Fig. 9. Bandwidth consumed per nodes.

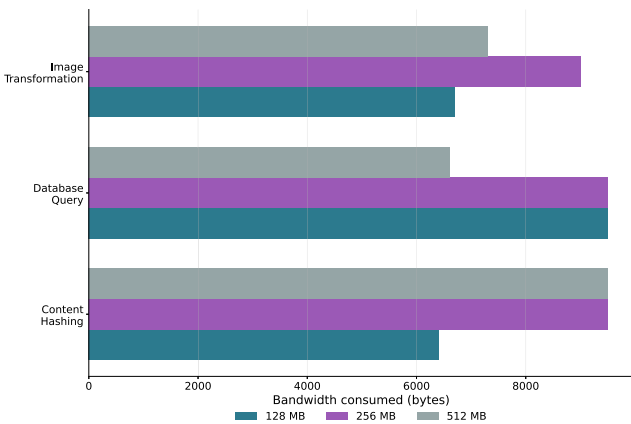


Fig. 10. Bandwidth consumed per node and memory value for each function type.

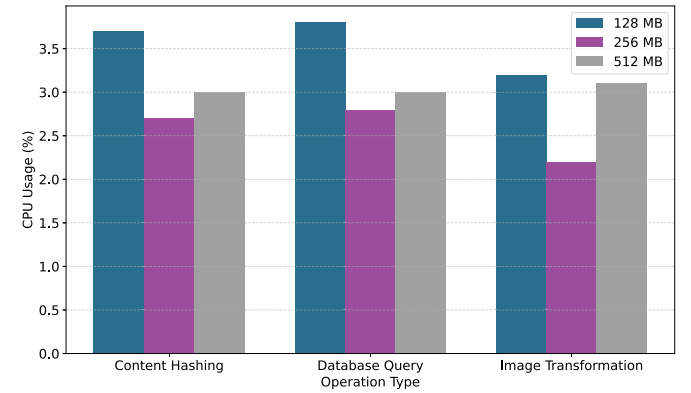


Fig. 11. CPU usage per memory range and function type.

as the number of nodes in the deployments increased, indicating an efficient utilization of the extra resources and good load balancing among supplier nodes. The usage in client nodes is also significantly lower (averaging between 0.30%-0.80% CPU) than in supplier nodes seeing as the latter are the ones satisfying the requests and running the OpenWhisk platform thus using more processing power.

Figs. 11 and 12 show that functions with 128MB of memory consumed the most CPU and memory resources, which we attribute to increased memory-disk swapping, which degraded performance. Despite this, the overall resource consumption for all memory allocations was deemed reasonable, ensuring that edge devices remain efficient for other functionalities while partaking in the FaaS@Edge network.

4.4. Request success rate

The request success rate serves as a key indicator of FaaS@Edge's system reliability by measuring both function submission and invocation completions. This metric directly reflects how well the resource discovery and scheduling algorithms perform in finding suitable edge nodes and managing function deployments. The evaluation revealed varying success rates across different function types, with image processing showing higher reliability while database operations faced occa-

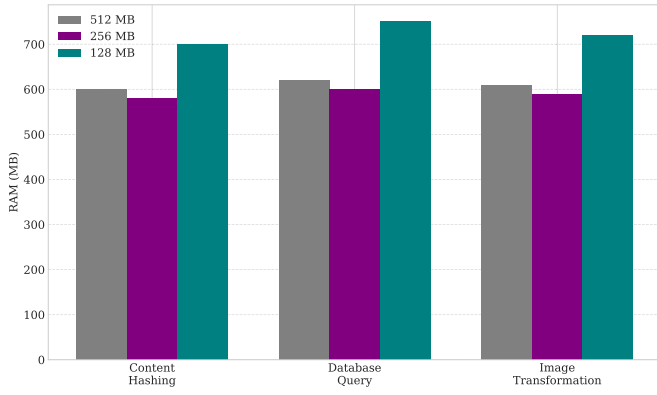


Fig. 12. RAM used per memory range and function type.

Table 1

Request success rate by function type and memory.

	Request Success Rate (%)	
	Submission	Invocation
<i>Function Type</i>		
Content Hashing	99.49	100.00
Database Query	95.16	94.98
Image Transformation	100.00	100.00
<i>Function Memory</i>		
128 MB	95.24	97.28
256 MB	99.49	98.73
512 MB	100.00	100.00
Total Requests	98.76	98.69

sional challenges due to node stability rather than resource constraints. The size of the memory allocation also influenced success rates, with larger allocations generally performing more reliably than smaller ones. These patterns highlight the system's overall effectiveness and specific areas of potential improvement in supporting edge computing workloads.

Table 1 presents success rates for FaaS workload functions and memory allocations during evaluation. Results show variations by function type and memory value, with image transformation reaching 100 % success while database queries encountered failures. Most invocation failures occurred after submission failures due to node crashes, not resource limits, as the allocation strategy distributed resources across requests. With failure rates of 1.24 % and 1.31 %, with one retry to another randomized offering node, the failure rates will be 0.0124^2 and 0.0131^2 respectively, yielding both a success rate of ≈ 99.98 %. The offering plan had minimal effect on bandwidth, CPU, and memory metrics, with resource updates maintaining performance levels across tests. This suggests the system balanced resource supply and demand independent of network size changes.

4.5. OpenWhisk local deployment

To evaluate the performance impact of concurrent function submissions on a local deployment of the OpenWhisk platform, we conducted an experiment focused on measuring the time required to create (i.e., submit) actions (serverless functions) under different concurrency levels. The test environment was a single-node OpenWhisk deployment configured on a virtualized edge host, with the function of Content Hashing. The creation of new actions was done with the standard command-line interface, i.e. the `wsk action create` command. The set of test are from a single submission to several concurrent action submissions. The results, shown in Fig. 13, present the average submission time across five rounds per levels of concurrency. Note that, to avoid starting a new pro-

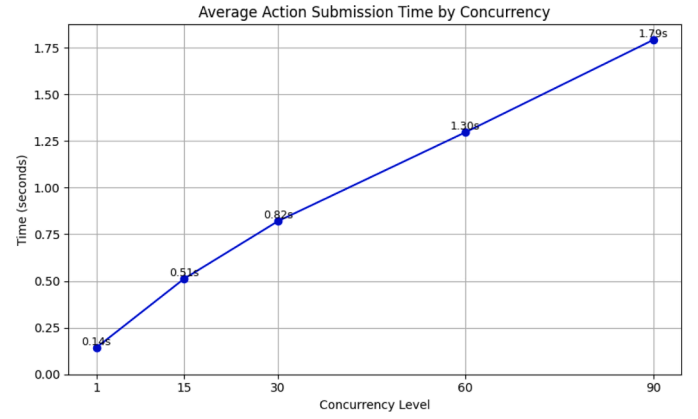


Fig. 13. OpenWhisk action creation time under concurrency.

cess, during regular FaaS@Edge operation, the system does not use the `wsk` tool, instead it uses the OpenWhisk REST API.

The chart highlights how submission latency scales with concurrency, revealing a near-linear increase in average time per round. Although OpenWhisk can handle concurrent submissions effectively at lower levels, performance degrades as the contention for internal resources increases. This suggests that in edge environments with limited control plane scalability, submission bursts could become a bottleneck, impacting responsiveness for deployment-intensive workflows. FaaS@Edge inherently decentralized architecture can mitigate this problem, as discussed in previous sections, leveraging the cluster of edge nodes to distribute the load of concurrent requests.

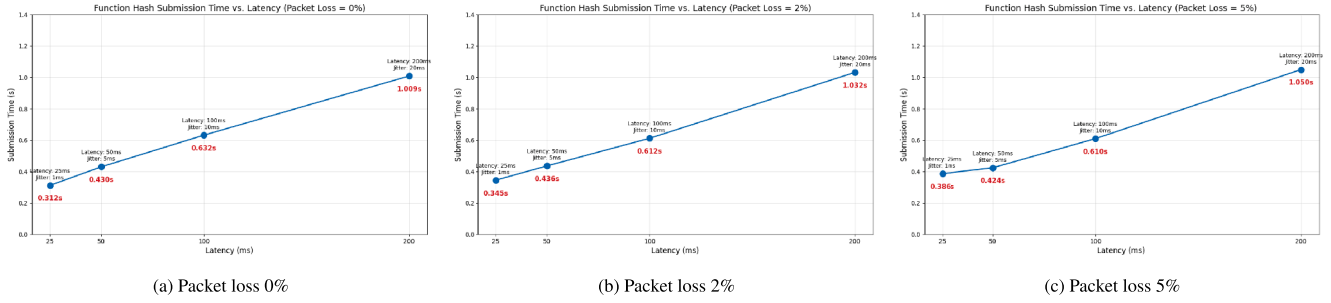
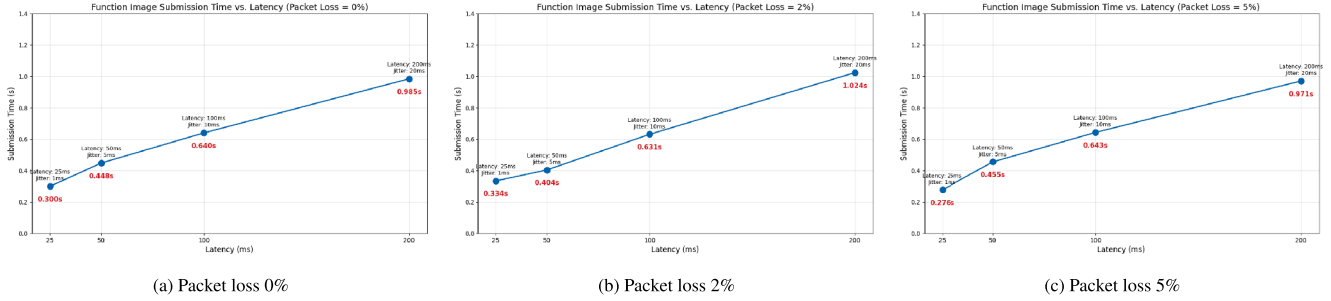
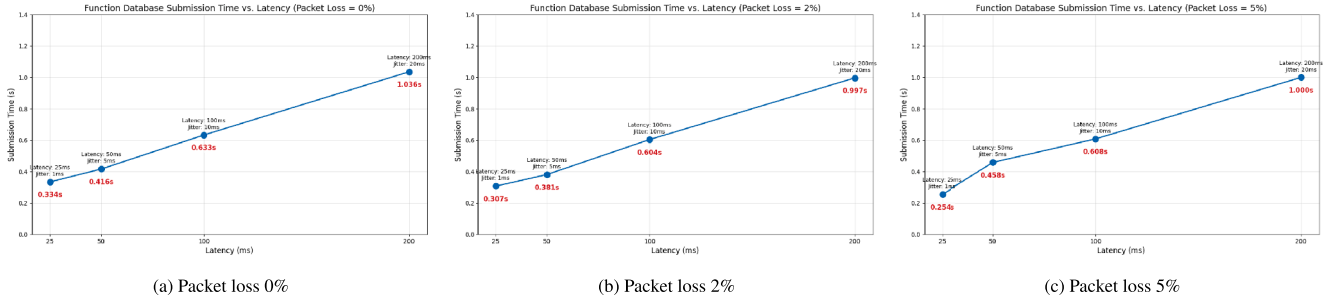
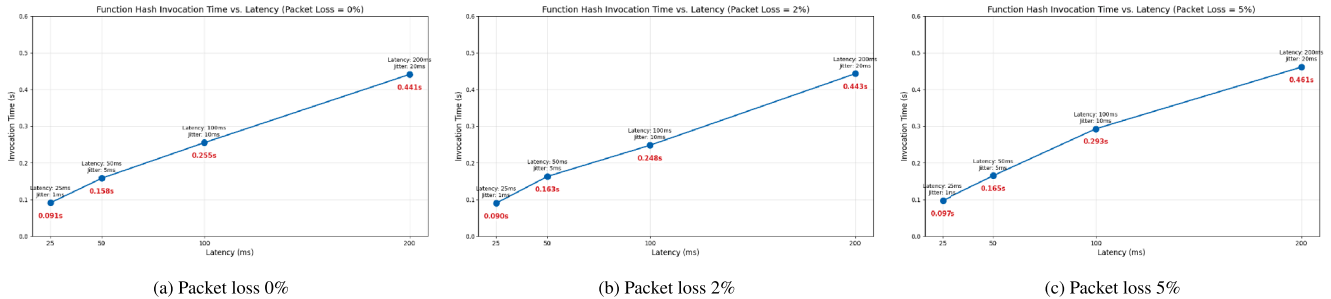
4.6. Variable network conditions

To evaluate the resilience of the FaaS@Edge system in more challenging deployment scenarios, it is important to determine how the variable condition of the network affects its operations. This can occur in scenarios of geographically dispersed nodes, or if the IPFS global network was used, but also in some edge configurations, as reported by Ali-Elden et al. [32].

The experimental setup consisted of two cluster deployments of 20 and 40 worker nodes, each node equipped with 2 vCPUs and 2 GB of RAM (suitable for being volunteered by users as resources to execute functions), in addition to a bootstrap node with identical hardware specifications. All worker nodes were uniformly configured with FaaS@Edge middleware, IPFS, and OpenWhisk. The network conditions were manipulated on each node using the Linux Traffic Control (tc) utility. The queueing discipline was established to introduce varying levels of the three network parameters under evaluation. The following experiments focus on the performance of the function submission and function invocation process, for the three workloads considered and discussed previously, involving key components of the system. The packet loss rates considered were: 0 %, 2 % and 5 %; latency values considered were: 25, 50, 100 and 200ms; with the corresponding jitter of ≈ 10 % of latency value. This resulted in 12 different combinations of network variability parameters. For each, the benchmark applications were employed similarly to the conditions in Section 4.1, and total average times are presented for each metric.

4.6.1. Variable network deployment A (20 nodes)

A deployment with variable networking conditions containing 20 nodes was evaluated, setting network parameters across all nodes. Figs. 14–16 illustrate how latency, jitter, and packet loss impact the execution time of the function submission operation. This operation involves all distributed components of the system architecture. For each type of function, after each node was set up, the submit function operation was called using the FaaS@Edge command line, and the overall

Fig. 14. Function *hash* submission time with variable latency, jitter and packet loss. (20 nodes).Fig. 15. Function *image* submission time with variable latency, jitter and packet loss. (20 nodes).Fig. 16. Function *database* submission time with variable latency, jitter and packet loss. (20 nodes).Fig. 17. Function *hash* invocation time with variable latency, jitter and packet loss. (20 nodes).

execution time was recorded, under the respective network conditions. The latency has a significant impact on the system, but the operation remains resilient to these variations.

Regarding the invocation time of the benchmark functions, Figs. 17–19 show the impact of latency, jitter, and packet loss on these benchmarks. This operation involves the local FaaS@Edge daemon to find the node where the requested function is deployed, followed by a request to the corresponding node to execute the function at the corresponding OpenWhisk runtime, concluding with the processing of the result. For each type of function, after each node was set up, the function was invoked using the FaaS@Edge command line and the overall execution time was recorded, under the respective network conditions.

Here, again, we can see that the latency and packet loss have an impact, mainly in hashing and database workloads. A more complex task, image transformation, is relatively less affected.

4.6.2. Variable network deployment B (40 nodes)

Another deployment with variable networking conditions containing 40 nodes was evaluated, with similar network parameters as the previous deployment. Figs. 20–22 show the impact of latency, jitter, and packet loss on a deployment with 40 nodes, with respect to submission time. The main difference in the results w.r.t to Deployment A occurs with higher latency (200ms) and packet loss (2% and more so with 5%). With a larger network, possibly requiring an extra hop to

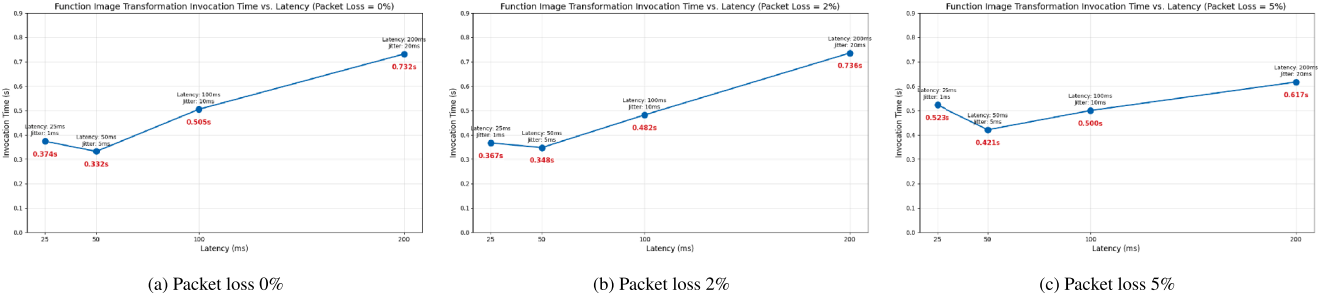


Fig. 18. Function *image transformation* invocation time with variable latency, jitter and packet loss. (20 nodes).

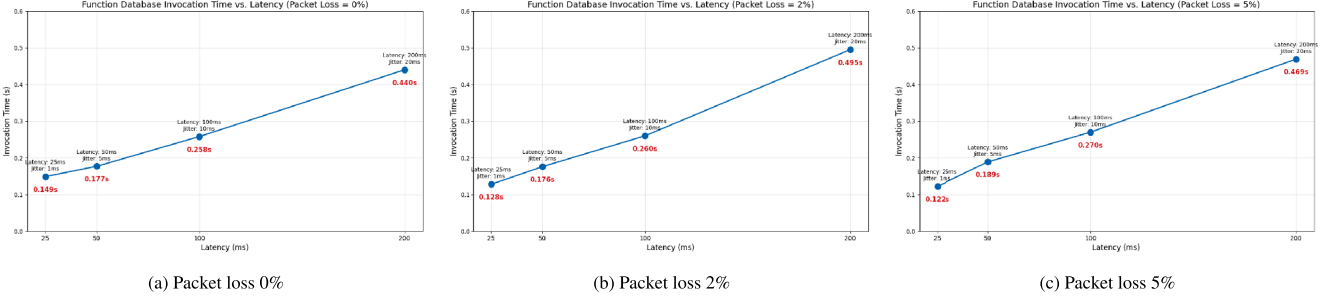


Fig. 19. Function *database* invocation time with variable latency, jitter and packet loss. (20 nodes).

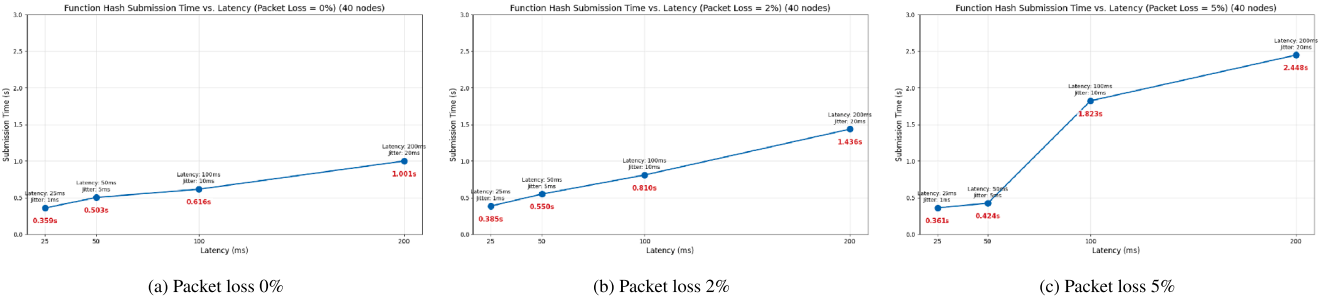


Fig. 20. Function *hash* submission time with variable latency, jitter and packet loss. (40 nodes).

reach other peers, higher latency and packet loss lead to increases in submission times (mainly for hashing and database).

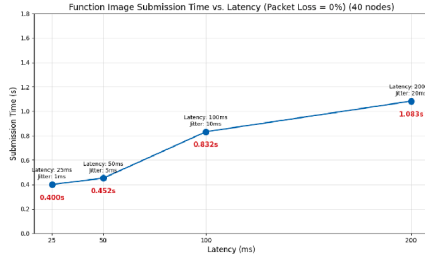
Figs. 23–25 show the impact of those variables on the invocation time between the benchmarks. Here, invocations times without packet loss scales almost linearly with the number of nodes, remaining overall at low values. When there is packet loss, and with higher latencies introduced, the invocation time exhibits a similar pattern. When there is packet loss, invocation time degrades. Nonetheless, globally, submission and invocation do not show differences between the two scenarios that could put scalability into question, which means that these core operations (submission and invocation) scale well with the number of nodes in the deployment.

4.7. Edge node performance

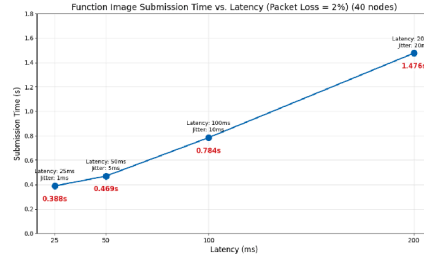
To validate that commodity edge devices can host the FaaS@Edge middleware, we deployed our system on a Raspberry Pi (RPI) 3 Model B (4 × ARM Cortex-A53 @ 1.2GHz, 1 GB LPDDR2 RAM, 10/100Mbps Ethernet, Ubuntu 20.04). The RPI node is deployed in a private network with 20 other more resourceful nodes. We measured the (i) memory footprint of this setup, (ii) response time of invoking a function with a number of increasing invocations and (iii) offers lookup execution time and bytes sent/received.

FaaS@Edge demonstrated efficient resource utilization, consuming only 315MB of the 907MB available to run both OpenWhisk (OW) and IPFS. This indicates that the framework is capable of running in resource-constrained edge devices, leaving more than 65% of memory for functions to be loaded and for other processes. Fig. 26(a) depicts the total invocation time of a minimal function previously submitted, from the client perspective, as the number of concurrent clients increases. It shows an increase from $\approx 1000ms$ at 1 concurrent invocation to over $4000ms$ at 20 concurrent invocations. This suggests that while the system performs well under low concurrency, higher concurrency levels may introduce significant latency, due to the limited processing power and memory of the Raspberry Pi. Currently, the person managing the node is responsible for setting up FaaS@Edge so that concurrency is kept within an acceptable limit.

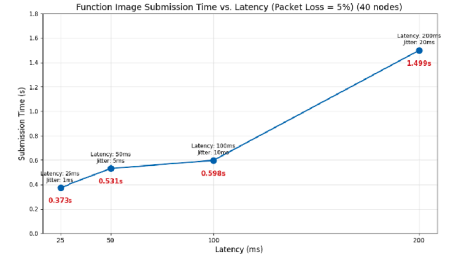
Regarding resource offer publishing and lookup, all nodes published an offer, resulting in 20x5 offer information stored in IPFS, corresponding to 100 small files. Fig. 26(b) shows the lookup time of 10 random CID and the average time to do so. The average time to retrieve a CID is approximately $209ms$, as indicated by the horizontal line. Most hashes exhibit times close to this average, with values ranging from around $200ms$ to just above $250ms$. This consistency suggests that the system performs reliably in locating CIDs also when resource-constrained edge devices are present in the network. Finally, Fig. 26(c) shows the received



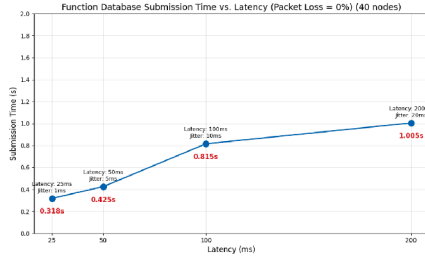
(a) Packet loss 0%



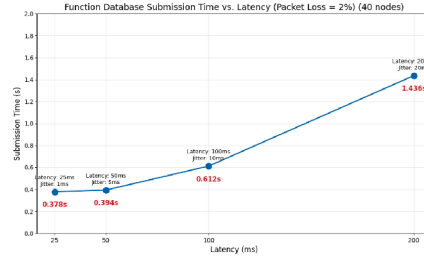
(b) Packet loss 2%



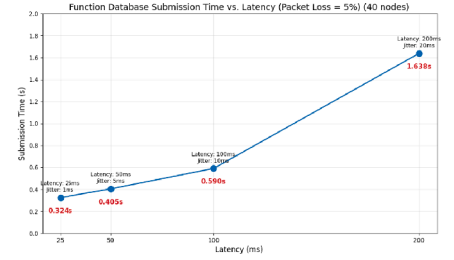
(c) Packet loss 5%

Fig. 21. Function *image* submission time with variable latency, jitter and packet loss. (40 nodes).

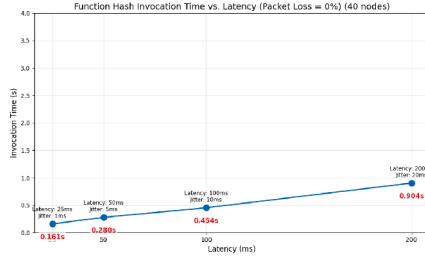
(a) Packet loss 0%



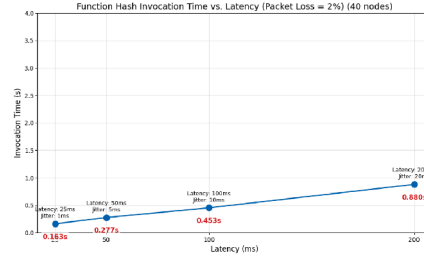
(b) Packet loss 2%



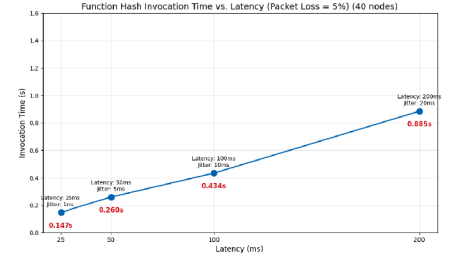
(c) Packet loss 5%

Fig. 22. Function *database* submission time with variable latency, jitter and packet loss. (40 nodes).

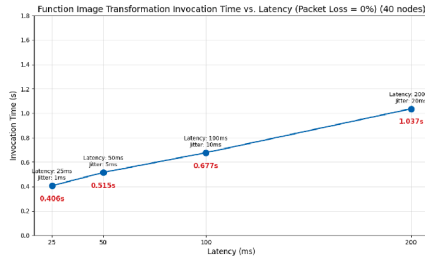
(a) Packet loss 0%



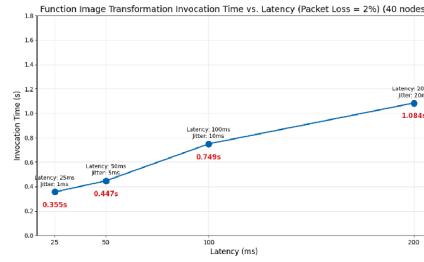
(b) Packet loss 2%



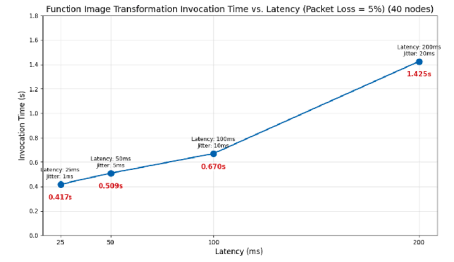
(c) Packet loss 5%

Fig. 23. Function *hash* invocation time with variable latency, jitter and packet loss. (40 nodes).

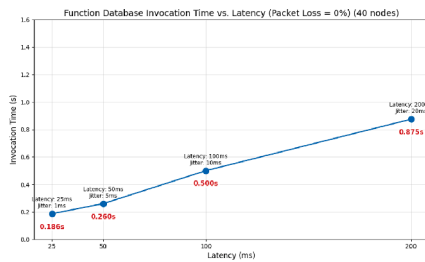
(a) Packet loss 0%



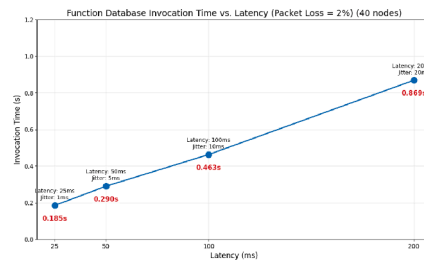
(b) Packet loss 2%



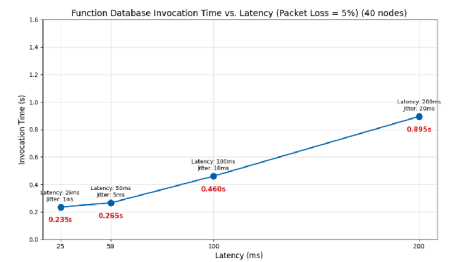
(c) Packet loss 5%

Fig. 24. Function *image* invocation time with variable latency, jitter and packet loss. (40 nodes).

(a) Packet loss 0%



(b) Packet loss 2%



(c) Packet loss 5%

Fig. 25. Function *database* invocation time with variable latency, jitter and packet loss. (40 nodes).

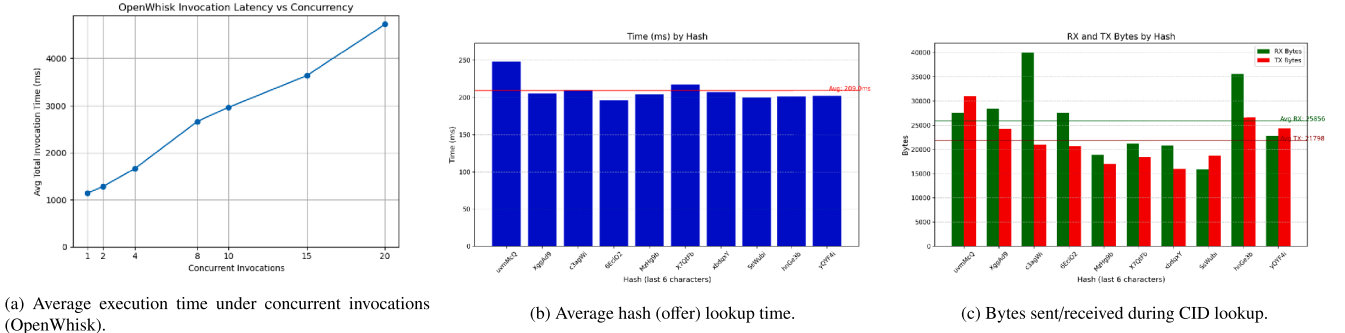


Fig. 26. Performance analysis on raspberry Pi 3: (a) OpenWhisk concurrency, (b-c) IPFS offer lookup.

(RX) and transmitted (TX) bytes for various hashes, providing information on the network traffic associated with retrieving CIDs. Although there are some variations for each CID, reflecting the nature of the IPFS algorithm to search for a given information, this does not seem to have an influence on the execution time of the offer lookup.

5. Related work and discussion

Applications organized as a set of stateless functions targeting the FaaS emerging paradigm were originally designed for cloud environments. However, this approach has increasingly been adopted in Edge-based environments, such as IoT systems, to bring computing closer to devices, thus minimizing latency and reducing energy and resource usage [33]. In general, these systems have the goal of adapting the meta-architecture of FaaS runtimes to resource-constrained hardware. Examples of this research include *tinyFaaS* [12] which uses the Constrained Application Protocol (CoAP), a lightweight Web Transfer Protocol, based on UDP, designed for use in constrained environments and used by *tinyFaaS* to allocate and invoke functions. Baresi and Mendonça [6] adapted the main components of OpenWhisk to run more efficiently on devices with limited computing capacities, showing savings in memory and latency compared to the base version of OpenWhisk. There is also a family of systems that use WebAssembly as an execution runtime [34–36], to run functions written or translated into WebAssembly. This method tries to address the performance and portability challenges in edge systems. Wasm offers a lightweight, sandboxed runtime with near-native execution speed and low startup latency. Its platform-agnostic binary format ensures consistent behaviour across diverse hardware and makes a potential fit for dynamic, low-latency workloads at the edge.

The research topic of orchestration of Edge-Cloud environments [37] aims to find ways to vertically offload tasks to a relatively small number of stable edge servers [38–40]. These types of architecture treat edge nodes as managed extensions of the datacenter, optimizing long-running jobs through global system-wide scheduling. Systems such as Picasso [38] adopt a network provider-centric model, in which the provider exercises complete control over the communication and edge infrastructure. Edge nodes are single-board computers, such as the Raspberry Pi, where a monitoring, execution, and proxy service are running to locally execute request or forward them to the next available node. Knative Edge [40] is designed to deploy serverless functions across both edge and cloud environments. It extends the Knative serverless platform to enable offloading requests from edge devices to the cloud.

Another more challenging approach was taken by Cicconetti et al. [37,41] assuming that devices have limited connectivity to a remote data center. This is also an assumption made by FaaS@Edge, where scheduling is done in a distributed way and independently from a central service provider. However, Cicconetti et al. assume that all VMs/containers required for the execution of the functions are already provisioned before the user starts sending their requests, missing the opportunity to a more dynamic and balanced environment.

Our work also crosses the boundaries of volunteer computing, where individuals contribute unused resources to form a distributed computational system. A well-known example is SETI@home [42], which uses the BOINC platform [43] to analyze radio signals for extraterrestrial intelligence. While BOINC is designed for specific applications, extensions like nuBOINC [44] expand its use to general cycle-sharing tasks. Caravela [45] is also a decentralized Edge Cloud system that uses volunteered resources, allowing users to deploy applications via Docker containers. Similarly to Picasso [38], Caravela's deployment artifact is also a container image. Peers can act as resource suppliers, buyers deploying containers, or traders managing resource offers within a defined region. Our current work fosters these principles and extend them to apply them in the context of serverless edge computing.

IPFS [16,30] is a decentralized peer-to-peer file system that integrates concepts from DHTs, block exchanges, version control, and self-certified file systems. It uses a Kademlia-based DHT to discover peers and locate content stored on specific nodes. Data in IPFS is divided into content-addressed chunks that form a Merkle DAG, allowing objects to be retrieved using the root of their DAG. The BitSwap protocol manages data distribution, with peers maintaining lists of desired and offered content chunks for exchange. Additionally, IPFS supports publish-subscribe notifications [46]. DeFaaS [25] targets Web3 dApps that are to be deployed as functions in multiple cloud providers. It uses the pub-sub functionality of IPFS to distribute events between different clouds. These features make IPFS suitable for distributed file systems, facilitating the exchange of FaaS deployment images and enabling decentralized resource discovery in edge networks through its DHT-based mechanisms.

WOW [34] is a WebAssembly runtime prototype for serverless edge computing, offering a lightweight alternative to traditional container runtimes. Using Apache OpenWhisk, the system enables developers to compile function code from multiple programming languages into WebAssembly, deployed through a modified interface. The architecture includes an Executor managing runtime binaries, an Invoker processing and routing execution requests, and a WebAssembly module containing function code. Experimental results indicate promising performance improvements in cold start times and memory usage for edge computing environments.

The previous systems address some of the aspects that we also tackle in our solution. However, none achieves the implementation of all aspects. IPFS focuses on content storage and distribution, which is highly important in P2P edge environments, but involves no computation execution by itself. Caravela uses a P2P network with similar capabilities as IPFS and introduces the execution of long-running container applications; it is not designed for FaaS deployments. Apache OpenWhisk is a framework for FaaS deployments, but it was not intentionally designed to maintain performance in an edge environment and does not feature content distribution. WOW focuses solely on the aspects of FaaS execution in edge computing nodes, abstracted from its integration in a distributed and decentralized network architecture. The system proposed

Table 2

Comparison of systems based on ability to do distributed scheduling, decisions based on resource offers, offloading to/from cloud, and FaaS execution.

System	Distributed Scheduling/Orchestration	Resource-aware	Cloud-Edge offloading	FaaS Execution
Knative Edge [40]	×	×	✓	✓
tinyFaaS [12]	×	×	×	✓
Serverledge [39]	×	×	✓	✓
Caravela [45]	✓	✓	×	×
WOW [34]	×	×	×	✓
E-Router [37]	✓	×	×	✓
Tasklet [47]	✓	✓	×	×
Picasso [38]	×	×	✓	×
DeFaaS[25]	✓	×	×	✓

by Bareri and Mendonça [6] also lacks the goal of having a decentralized system, based on a peer-to-peer overlay that connects the provider nodes through peer-to-peer content discovery service, also leveraged for resource discovery.

Table 2 provides a comparative analysis of selected systems relevant to edge computing and Function-as-a-Service. The comparison focuses on four architecture criteria: support for distributed scheduling or orchestration; resource awareness in decision-making; integration of cloud-edge offloading mechanisms; and the ability to execute FaaS workloads. Distributed Scheduling/Orchestration refers to whether the system supports decentralized mechanisms for assigning and managing tasks across multiple nodes, as opposed to relying on a centralized controller. Resource-aware indicates whether the system takes into account node-level resource constraints (e.g. CPU, memory, bandwidth) when making scheduling or execution decisions. Cloud-Edge Offloading represents the system's ability to shift computational workloads between cloud data centers and edge nodes, based on current network or workload conditions. FaaS execution refers to whether the system supports the execution of stateless, short-lived functions.

In summary, unlike existing platforms that each address only a subset of requirements, such as offloading and FaaS execution (e.g., Knative Edge, Serverledge), distributed scheduling and resource awareness (e.g., Caravela, Tasklet), or resource-aware offloading (e.g., Picasso), FaaS@Edge integrates decentralized orchestration, explicit resource-aware placement, and native FaaS invocation into a single framework. While we primarily focus on edge-to-edge function placement, FaaS@Edge also partially supports hybrid deployments by allowing unserved requests or excess load to be forwarded to cloud instances of OpenWhisk. The proposed system provides a solution for serverless deployments in resource-constrained dynamic edge environments.

6. Conclusion

In this study, we present FaaS@Edge, a decentralized framework that implements Function-as-a-Service (FaaS) capabilities in edge computing environments by orchestrating user function deployment across distributed edge nodes through Apache OpenWhisk. Empirical analysis reveals that while the middleware introduces substantial initial overhead, approximately doubling the latency for function submission compared to traditional local OpenWhisk deployments, this does not increase significantly with scale and network variability, and the subsequent invocation latencies remain consistent with baseline measurements. This performance characteristic proves advantageous given the typical operational pattern in which the invocation frequency significantly exceeds the submission events. Performance metrics, including bandwidth consumption, CPU utilization, and memory footprint, demonstrate that the system operates within the resource constraints of the edge devices. Furthermore, FaaS@Edge exhibits robust reliability in both the function deployment and execution phases, with high success rates across distributed environments.

Future research directions cover three main objectives. First, we aim to expand the deployment methodologies beyond source code files and

custom runtimes to include Docker containers and binary-compatible executables. Second, we plan to implement co-located function submission capabilities, offering users the flexibility to either consolidate multiple functions on a single supplier node or distribute them across the network. Although each node is already capable of receiving concurrent requests, the user can benefit from allocating a set of functions in the same node, if resource offers are available, and reduce latency to access databases or files at the target node. This enhancement would also facilitate simultaneous invocations and enable the implementation of processing pipelines, analogous to OpenWhisk's 'sequence' functionality. Finally, we want to exploit locality in the overlay network, aiming that each client schedules functions in nodes preferably closer (e.g., geographically, latency-wise). This will enable managing sets of nearby located nodes using alternative heuristics (e.g. round-robin, and following co-prime logic), as OpenWhisk employs in local deployments.

CRediT authorship contribution statement

Catarina Gonçalves: Writing – review & editing, Writing – original draft, Software, Investigation; **José Simão:** Writing – review & editing, Writing – original draft, Validation, Software, Investigation, Conceptualization; **Luís Veiga:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by national funds through FCT, Fundação para a Ciência e a Tecnologia, under projects UIDB/50021/2020 (DOI:10.54499/UIDB/50021/2020) and 2023.16994.ICDT, and through programme Lisboa 2030, under project LISBOA2030-FEDER-00674200. This work was supported by the CloudStars project, funded by the European Union's Horizon research and innovation program under grant agreement number 101086248. In this context, this work was partially developed while Luís Veiga was a Visiting Researcher with the Hybrid Cloud Computing Group at IBM Research Europe – Zurich.

References

- [1] I. Baldini, P. Castro, K. Chang, P. Cheng, S. Fink, V. Ishakian, N. Mitchell, V. Muthusamy, R. Rabbah, A. Slominski, P. Suter, Serverless Computing: Current Trends and Open Problems, Springer Singapore, Singapore, 2017, pp. 1–20. https://doi.org/10.1007/978-981-10-5026-8_1

- [2] S. Fouladi, R.S. Wahby, B. Shacklett, K.V. Balasubramaniam, W. Zeng, R. Bhalerao, A. Sivaraman, G. Porter, K. Winstein, Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads, 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17), USENIX Association, Boston, MA, 2017, pp. 363–376. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/fouladi>
- [3] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, et al., Cloud programming simplified: a Berkeley view on serverless computing, *arXiv preprint arXiv:1902.03383* (2019).
- [4] P. Raith, S. Nastic, S. Dustdar, Serverless edge computing-where we are and what lies ahead, *IEEE Internet Comput.* 27 (3) (2023) 50–64. <https://doi.org/10.1109/MIC.2023.3260939>
- [5] S. Fouladi, R.S. Wahby, B. Shacklett, K.V. Balasubramaniam, W. Zeng, R. Bhalerao, A. Sivaraman, G. Porter, K. Winstein, Encoding, fast and slow: low-latency video processing using thousands of tiny threads, in: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17), USENIX Association, Boston, MA, 2017, pp. 363–376. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/fouladi>.
- [6] L. Baresi, D. Filgueira Mendonça, Towards a serverless platform for edge computing, in: 2019 IEEE International Conference on Fog Computing (ICFC), 2019, pp. 1–10. <https://doi.org/10.1109/ICFC.2019.00008>
- [7] C. Systems, Fog Computing and the Internet of Things: Extend the Cloud to Where the Things Are, White Paper, 2016.
- [8] E. Union, Study on the Economic Potential of Far Edge Computing in the Future Smart Internet of Things. <https://op.europa.eu/s/zv4>.
- [9] A. Mathur, D.J. Beutel, P.P.B. de Gusmao, J. Fernandez-Marques, T. Topal, X. Qiu, T. Parcollet, Y. Gao, N.D. Lane, On-device federated learning with flower, *arXiv preprint arXiv:2104.03042* (2021).
- [10] F. Freitag, L. Wei, C.-H. Liu, M. Selimi, L. Veiga, Server-side adaptive federated learning over wireless mesh network, in: International Conference on Information Technology & Systems, Springer, 2023, pp. 289–298.
- [11] E. Gelenbe, Electricity consumption by ICT: facts, trends, and measurements, *Ubiquity* 2023 (August) (2023). <https://doi.org/10.1145/3613207>
- [12] T. Pfandzelter, D. Bermbach, tinyFaaS: a lightweight faas platform for edge environments, in: 2020 IEEE International Conference on Fog Computing (ICFC), IEEE, 2020, pp. 17–24.
- [13] O. Ascigil, A.G. Tasiopoulos, T.K. Phan, V. Sourlas, I. Psaras, G. Pavlou, Resource provisioning and allocation in function-as-a-service edge-clouds, *IEEE Trans. Serv. Comput.* 15 (4) (2021) 2410–2424.
- [14] A. Antelmi, G. D'Ambrosio, A. Petta, L. Serra, C. Spagnuolo, A volunteer computing architecture for computational workflows on decentralized web, *IEEE Access* 10 (2022) 98993–99010.
- [15] A. Foundation, Apache Openwhisk. Open Source Serverless Cloud Platform, <https://openwhisk.apache.org/>.
- [16] L. Baldof, M. Korczyński, O. Ascigil, N.V. Keizer, G. Pavlou, B. Scheuermann, M. Król, The cloud strikes back: investigating the decentralization of IPFS, in: Proceedings of the 2023 ACM on Internet Measurement Conference, IMC '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 391–405. <https://doi.org/10.1145/3618257.3624797>
- [17] C. Gonçalves, J. Simão, L. Veiga, FaaS@Edge: Bringing Function-as-a-Service to Voluntary Computing at the Edge, *Economics of Grids, Clouds, Systems, and Services*, M. Naldi, K. Djemame, J. Altmann, J.A. Bañares, Springer Nature Switzerland, Cham, 2025, pp. 28–42. https://doi.org/10.1007/978-3-031-81226-2_4
- [18] S.K. Mohanty, G. Premsankar, M. Di Francesco, et al., An evaluation of open source serverless computing frameworks, *CloudCom* 2018 (2018) 115–120.
- [19] J. Wen, Y. Liu, Z. Chen, J. Chen, Y. Ma, Characterizing commodity serverless computing platforms, *J. Software: Evol. Process* (2021) e2394.
- [20] K. Cao, Y. Liu, G. Meng, Q. Sun, An overview on edge computing research, *IEEE Access* 8 (2020) 85714–85728.
- [21] U.C. Özyar, A. Yurdakul, A decentralized framework with dynamic and event-driven container orchestration at the edge, in: 2022 IEEE International Conferences on Internet of Things (iThings) and IEEE Green Computing & Communications (GreenCom) and IEEE Cyber, Physical & Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics), IEEE, 2022, pp. 33–40.
- [22] C.-H. Hong, B. Varghese, Resource management in fog/edge computing: a survey on architectures, infrastructure, and algorithms, *ACM Computing Surveys (CSUR)* 52 (5) (2019) 1–37.
- [23] A. Sarma, J. Yoo, J.K. Sojan, U. Ramachandran, M. Lee, FEO: efficient resource allocation for FaaS at the edge, in: Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems, DEBS '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 78–89. <https://doi.org/10.1145/3629104.3666033>
- [24] R. Jin, Q. Yang, M. Zhao, When edge meets FaaS: opportunities and challenges, in: 2023 IEEE International Conference on Edge Computing and Communications (EDGE), 2023, pp. 23–25. <https://doi.org/10.1109/EDGE60047.2023.00016>
- [25] R. Karanjai, L. Xu, L. Chen, N. Diallo, W. Shi, Decentralized FaaS over multi-clouds with blockchain based management for supporting emerging applications, in: Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, SAC '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 122–130. <https://doi.org/10.1145/3605098.3636029>
- [26] A. Sathiaselalan, M. Selimi, C. Molina, A. Lertsinsruttavee, L. Navarro, F. Freitag, F. Ramos, R. Baig, Towards decentralised resilient community clouds, in: Proceedings of the 2nd Workshop on Middleware for Edge Clouds & Cloudlets, MECC '17, Association for Computing Machinery, New York, NY, USA, 2017. <https://doi.org/10.1145/3152360.3152363>
- [27] J. Switzer, G. Marciano, R. Kastner, P. Pannuto, Junkyard computing: repurposing discarded smartphones to minimize carbon, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Association for Computing Machinery, New York, NY, USA, 2023, p. 400–412. <https://doi.org/10.1145/3575693.3575710>
- [28] A. Gupta, H. Richardson, Software Carbon Intensity (SCI) Specification Project. <https://greensoftware.foundation/articles/software-carbon-intensity-sci-specification-project>.
- [29] A. Schmidt, G. Stock, R. Ohs, L. Gerhorst, B. Herzog, T. Hönig, Carbond: an operating-system daemon for carbon awareness, in: Proceedings of the 2nd Workshop on Sustainable Computer Systems, HotCarbon '23, Association for Computing Machinery, New York, NY, USA, 2023. <https://doi.org/10.1145/3604930.3605707>
- [30] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, B. Gipp, Y. Psaras, Design and evaluation of IPFS: a storage layer for the decentralized web, in: Proceedings of the ACM SIGCOMM 2022 Conference, SIGCOMM '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 739–752. <https://doi.org/10.1145/3544216.3544232>
- [31] V. Yussupov, U. Breitenbücher, F. Leymann, C. Müller, Facing the inplanned migration of serverless applications: a study on portability problems, solutions, and dead ends, in: Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing, UCC'19, Association for Computing Machinery, New York, NY, USA, 2019, p. 273–283. <https://doi.org/10.1145/3344341.3368813>
- [32] A. Ali-Eldin, B. Wang, P. Shenoy, The hidden cost of the edge: a performance comparison of edge and cloud latencies, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21, Association for Computing Machinery, New York, NY, USA, 2021. <https://doi.org/10.1145/3458817.3476142>
- [33] G.A.S. Cassel, V.F. Rodrigues, R. da Rosa Righi, M.R. Bez, A.C. Nepomuceno, C. André da Costa, Serverless computing for internet of things: a systematic literature review, *Future Gen. Comput. Syst.* 128 (2022) 299–316. <https://doi.org/https://doi.org/10.1016/j.future.2021.10.020>
- [34] P. Gackstatter, P.A. Frangoudis, S. Dustdar, Pushing serverless to the edge with webassembly runtimes, in: 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), IEEE, 2022, pp. 140–149.
- [35] A. Hall, U. Ramachandran, An execution model for serverless functions at the edge, in: Proceedings of the International Conference on Internet of Things Design and Implementation, IoTDI '19, Association for Computing Machinery, New York, NY, USA, 2019, p. 225–236. <https://doi.org/10.1145/3302505.3310084>
- [36] G. De Palma, S. Giallorenzo, J. Mauro, M. Trentin, G. Zavattaro, Web assembly at the edge: benchmarking a serverless platform for private edge cloud systems, *IEEE Internet Comput.* 28 (6) (2024) 37–44. <https://doi.org/10.1109/MIC.2024.3513035>
- [37] C. Ciconetti, M. Conti, A. Passarella, A decentralized framework for serverless edge computing in the internet of things, *IEEE Trans. Netw. Serv. Manage.* 18 (2) (2021) 2166–2180. <https://doi.org/10.1109/TNSM.2020.3023305>
- [38] A. Lertsinsruttavee, A. Ali, C. Molina-Jimenez, A. Sathiaselalan, J. Crowcroft, Pi-Casso: a lightweight edge computing platform, in: 2017 IEEE 6th International Conference on Cloud Networking (CloudNet), 2017, pp. 1–7. <https://doi.org/10.1109/CloudNet.2017.8071529>
- [39] G.R. Russo, T. Mannucci, V. Cardellini, F.L. Presti, Serverledge: decentralized function-as-a-service for the edge-cloud continuum, in: 2023 IEEE International Conference on Pervasive Computing and Communications (PerCom), 2023, pp. 131–140. <https://doi.org/10.1109/PERCOM56429.2023.10099372>
- [40] E. Simion, Y. Wang, H.-I. Tai, U. Odyurt, Z. Zhao, Towards seamless serverless computing across an edge-cloud continuum, in: Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing, UCC '23, Association for Computing Machinery, New York, NY, USA, 2024. <https://doi.org/10.1145/3603166.3632537>
- [41] C. Ciconetti, M. Conti, A. Passarella, An architectural framework for serverless edge computing: design and emulation tools, in: 2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), 2018, pp. 48–55. <https://doi.org/10.1109/CloudCom2018.2018.00024>
- [42] D.P. Anderson, J. Cobb, E. Korpela, M. Lebofsky, D. Werthimer, SETI@ home: an experiment in public-resource computing, *Commun ACM* 45 (11) (2002) 56–61.
- [43] D.P. Anderson, BOINC: A platform for volunteer computing, *J. Grid Comput.* 18 (1) (2020) 99–122.
- [44] J.N. Silva, L. Veiga, P. Ferreira, Nuboinc: boinc extensions for community cycle sharing, in: 2008 Second IEEE International Conference on Self-Adaptive and Self-Organizing Systems Workshops, IEEE, 2008, pp. 248–253.
- [45] A. Pires, J. Simão, L. Veiga, Distributed and decentralized orchestration of containers on edge clouds, *J. Grid Comput.* 19 (3) (2021) 36. <https://doi.org/10.1007/s10723-021-09575-x>
- [46] J. Antunes, D. Dias, L. Veiga, Pulsarcas: scalable, reliable pub-sub over P2P nets, in: Z. Yan, G. Tyson, D. Koutsonikolas (Eds.), IFIP Networking Conference, IFIP Networking 2021, Espoo and Helsinki, Finland, June 21–24, 2021, IEEE, 2021, pp. 1–6. <https://doi.org/10.23919/IFIPNETWORKING52078.2021.9472799>
- [47] J. Edinger, D. Schäfer, C. Krupitzer, V. Raychoudhury, C. Becker, Fault-avoidance strategies for context-aware schedulers in pervasive computing systems, in: 2017 IEEE International Conference on Pervasive Computing and Communications (PerCom), 2017, pp. 79–88. <https://doi.org/10.1109/PERCOM.2017.7917853>